- *Security*—The application lacks access restrictions. If deployed, anyone could interact with it, which could lead to unexpected API costs.
- *Improved tooling*—Our current setup uses separate frontend (React) and backend (Express.js) components. A more integrated framework could streamline this process.

We'll address these problems as the chapters progress and especially focus on managing memory and context, streaming, and security considerations. But we can make some enhancements to the base project right away by migrating the backend framework from Express.js to Next.js., a powerful React framework. This shift streamlines the deployment process and has the potential to improve application performance through features like server-side rendering and static site generation. The most compelling benefit lies in the synergy with our existing React codebase. Next.js integrates seamlessly with React, allowing a more cohesive and efficient development environment.

While Next.js is a powerful tool, it's not a one-size-fits-all solution. The best framework choice depends on your specific project requirements and team expertise. However, for complex and dynamic LLM-powered web applications, Next.js offers a compelling set of features that can significantly enhance performance, maintainability, and security.

2.7 Migrating the app to Next.js

To level up our web application and begin to address some of its key limitations, our next step is to migrate to Next.js, a full-stack framework that will simplify our architecture and unlock key features. This isn't about replacing tools; it's about adopting a more cohesive framework to build a more robust application. Migrating to Next.js will allow us to simplify our codebase while maintaining functionality and to use built-in optimizations for performance and deploying for production. It will also unlock built-in server-side rendering and static site generation.

Most of the original project requirements likely remain unchanged for now. The primary difference lies in the build tools used. Previously, we used Vite, a development server and build tool for modern web applications. But with Next.js, you don't necessarily need a separate build tool like Vite. Next.js includes an integrated build system optimized for server-side rendering and static site generation. This simplifies the development process and provides built-in functionalities, such as code splitting and optimization in various environments.

Next.js also includes a command-line interface with several commands for managing your project. Some common commands include

- `next dev`—Starts the development server, allowing you to work on your application with features like hot module replacement
- `next build`—Builds your application for production, optimizing the code and assets for deployment